

L5

SELECȚIA ÎNREGISTRĂRILOR DIN BAZELE DE DATE

1. Obiective

1. Comanda SELECT
 - Sintaxă;
 - Exemple de aplicare.

2. Considerații teoretice

2.1 Comanda **SELECT**

Comanda SELECT creează o mulțime de selecție.

În teoria mulțimilor o mulțime $(a_1, a_2, \dots, a_n)$ de obiecte distincte poartă numele de *n-uplu*.

Valoarea n este denumită *cardinalul* mulțimii iar un element a_i al acesteia se numește *uplu* (eng. *tuple*).

O mulțime de selecție poate conține un obiect, mai multe obiecte sau niciunul. Elementele dintr-o mulțime de selecție conțin numai câmpurile indicate în comanda de creare a acesteia. Din acest punct de vedere se poate considera că SELECT crează un subtabel conținând numai informațiile specificate. Câmpurile înregistrărilor mulțimii de selecție pot proveni dintr-un tabel sau din mai multe tabele legate.

SELECT - este comandă cea mai utilizată;
este folosită pentru obținerea datelor din bazele de date.

Comanda **SELECT** are formatul general:

SELECT [**DISTINCT**] coloana1 [, coloana2]

FROM table_1[,tabel_2, ...]

[**WHERE** condiții]

[**GROUP BY** listă-coloane]

[**HAVING** conditii]

[**ORDER BY** listă-coloane [**ASC** | **DESC**]]

Dintre cele cinci clauze ale comenzii **SELECT** numai clauza FROM este **obligatorie**. Fiecare dintre clauze are la rândul ei reguli și parametri pentru construcție, făcând din SELECT

cea mai complexă comandă a limbajului SQL. În frazele SELECT șirurile de caractere se pun între caractere ' (apostrof).

```
SELECT [ALL | DISTINCT]
{[nume-tabela.]* |
expr [sinonim], expr [sinonim],...}
FROM nume-tabelă [@ legatură][sinonim],...
nume-tabelă [@ legatură][sinonim],
[WHERE conditie]
[ CONNECT BY conditie [START WITH conditie] ]
[ GROUP BY { expr, expr.. | CUBE ( expr, expr.. ) | ROLLUP ( expr, expr ) } ]
[HAVING conditie]
[ {UNION | INTERSECT | MINUS} SELECT...]
[ ORDER BY {expr | număr-pozitie} [ASC | DESC]
{expr | număr-pozitie}[ASC | DESC],...
[ FOR UPDATE OF nume-col, nume-col,...[NOWAIT] ];
```

Clauzele comenzii trebuie utilizate în ordinea specificată în sintaxă, excepție făcând clauzele **CONNECT BY**, **START WITH**, **GROUP BY** și **HAVING** (care pot fi specificate în orice ordine). Clauzele **ORDER BY** și **FOR UPDATE OF** pot fi schimbate între ele. Clauza **ALL** determină afișarea tuturor rândurilor rezultate în urma cererii, spre deosebire de clauza **DISTINCT** care determină eliminarea duplicatelor, afișând doar rândurile distincte. Utilizarea caracterului asterisc (*) are ca efect selectarea tuturor coloanelor din tabela specificată prin clauza **FROM**, în ordinea în care au fost definite la creare.

În situația finală, fiecare expresie formulată în comandă devine un nume de coloană. Totodată, orice *sinonim*, dacă este specificat, este folosit în scopul etichetării expresiei precedente din tabela afișată. Dacă *sinonimul* conține blankuri sau caractere speciale cum sunt “+” și ”-“, trebuie incluse între apostrofuri.

Pentru a evita ambiguitatea, în cazul în care tabelele conțin coloane cu același nume, este necesară calificarea coloanelor cu numele tabeli.

Identificarea tabelor în care trebuie căutate datele corespunzătoare coloanelor specificați se face prin specificarea numelor lor după clauza **FROM**. În locul numelor de tabele se pot folosi sinonimele acestora.

Clauza **WHERE** specifică o condiție care este folosită pentru a selecta rândurile.

Clauza **CONNECT BY** indică faptul că rândurile formează o structură arborescentă. Prin această clauză sunt definite relațiile necesare pentru a conecta rândurile tabeli într-un arbore. Operatorul **PRIOR** folosit înaintea uneia din cele două părți ale condiției, definește nodul părinte iar în cealaltă parte nodul fiu. Clauza **START WITH**, prin specificarea unei condiții care trebuie satisfăcută, stabilește rândul folosit ca rădăcină a arborelui. Dacă se omite, comanda **SELECT** va returna o serie de arbori începând cu fiecare rând selectat. Existența clauzei **CONNECT BY** într-o comandă **SELECT** permite utilizarea pseudocolonei **LEVEL**, care returnează valoarea 1 pentru nodul rădăcină, 2 pentru fiii nodului rădăcină, 3 pentru nepoți etc.

Clauzele **GROUP BY** și **HAVING** determină afișarea unor informații sintetice despre grupuri de rânduri care au aceeași valoare în una sau mai multe coloane. Aceste coloane sunt, în general, funcții de grup.

ROLLUP activează o comandă **SELECT** pentru a calcula mai multe niveluri de subtotaluri dintr-un grup specificat de dimensiuni. Calculează de asemenea și un total general. **ROLLUP** este o extensie simplă a clauzei **GROUP BY**, deci sintaxa este foarte ușor de folosit. Extensia **ROLLUP**

este foarte eficientă și nu îngreunează o cerere. Rolul extensiei ROLLUP este foarte clar: crează subtotaluri care pornesc de la cel mai detaliat nivel până la un total general după gruparea care a fost precizată în clauza ROLLUP.

CUBE acționează asupra unui grup specificat de coloane și crează subtotaluri pentru toate combinațiile posibile între acestea. Dacă s-a specificat de exemplu: CUBE (timp,regiune, department), rezultatul va include toate valorile care ar fi incluse într-un ROLLUP plus combinații adiționale.

Pentru a combina rezultatele a două comenzi SELECT într-un singur rezultat, se folosesc operatorii UNION, INTERSECT și MINUS. **UNION** returnează rezultatele obținute de la fiecare cerere în parte, **INTERSECT** returnează doar rezultatele comune celor două cereri iar **MINUS** returnează rezultatele obținute de la prima cerere și care nu apar în urma celei de a doua selecții.

Pentru a putea folosi aceste clauze este necesar ca numărul și tipul coloanelor selectate de fiecare comandă **SELECT** să fie aceleași, lungimile lor putând fi diferite. Dacă sunt combinate mai mult de două comenzi **SELECT**, ele vor fi evaluate de la stînga la dreapta. Pentru a schimba ordinea de evaluare pot fi folosite parantezele. Totodată, cei trei operatori impun utilizarea cuvântului **DISTINCT** în toate comenzile **SELECT**.

Clauza **ORDER BY** specifică ordinea în care trebuie returnate rîndurile distincte ale unei table.

Clauza **FOR UPDATE** determină blocarea rîndurilor selectate ale tablei astfel încât acestea nu vor mai putea fi actualizate de alți utilizatori până la deblocarea lor cu una din comenzile **COMMIT** sau **ROLL BACK**.

Comanda **SELECT ... FOR UPDATE** trebuie urmată de una sau mai multe comenzi **UPDATE ... WHERE**. Dacă se folosește clauza **NOWAIT**, selecția este considerată terminată chiar dacă rîndurile selectate de **FOR UPDATE** nu pot fi blocate deoarece alt utilizator lucrează cu ele.

2.1.1 Utilizarea clauzei FROM

Pentru a selecta una sau mai multe tabele și pentru a specifica, eventual, identificatorul proprietarului și legătura cu rețeaua se folosește secvența:

SELECT ...
FROM [ident-proprietar] tabela [@LINK],...;

Exemple:

1) Să se selecteze toate coloanele din tabela SALARIAȚI.

SQL> SELECT * FROM SALARIAȚI; sau

SQL> SELECT ALL FROM SALARIAȚI;

```
MARCA NUME FUNCT CODD SALA VENS CODS
1111 AVRAM ION VÂNZATOR 100000 21200 1000 1000
1222 BARBU DAN VÂNZATOR 120000 20750 2000 1000
1000 COMAN RADU ȘEF DEP 130000 35000 2500 1000
3500 DAN ION VÂNZATOR 160000 24500 3550 2500
2500 VLAD VASILE ȘEF DEP 160000 36500 1500 2500
3700 MANU DAN VÂNZATOR 160000 27500 2500 2500
2650 VLAD ION VÂNZATOR 120000 25060 3500 1000
```

7 records selected.

2) Să se selecteze coloanele MARCA, NUME, SALA, VENS din tabela SALARIAȚI.

SQL> SELECT MARCA,NUME,SALA,VENS
2 FROM SALARIAȚI;

MARCA NUME SALA VENS
 1111 AVRAM ION 21200 1000
 1222 BARBU DAN 20750 2000
 1000 COMAN RADU 35000 2500
 3500 DAN ION 24500 3550
 2500 VLAD VASILE 36500 1500
 3700 MANU DAN 27500 2500
 2650 VLAD ION 25060 3500

7 records selected.

3) Să se selecteze coloana NUME din tabela SALARIAȚI

**SQL> SELECT NUME
 2 FROM SALARIAȚI;**

NUME
 AVRAM ION
 BARBU DAN
 COMAN RADU
 DAN ION
 VLAD VASILE
 MANU DAN
 VLAD ION

7 records selected.

4) Să se selecteze coloana FUNCT din tabela SALARIAȚI în variantele utilizării și neutilizării clauzei DISTINCT.

**SQL> SELECT FUNCT AFIȘARE_FUNCTIE
 2 FROM SALARIAȚI;**

AFISARE_FUNCTIE
 VÂNZATOR
 VÂNZATOR
 ȘEF DEP
 VÂNZATOR
 ȘEF DEP
 VÂNZATOR
 VÂNZATOR

7 records selected.

**SQL> SELECT DISTINCT FUNCT
 2 AFIȘARE_DISTINCT_FUNCTIE
 3 FROM SALARIAȚI;**

AFISARE_DISTINCT_FUNCTIE
 VÂNZATR
 O ȘEF DEP

2 records selected.

3. Desfășurarea lucrării

Exemplu de utilizare:

Tabela **STUDENT**

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1234	POPA MARCEL	1	114A	12-03-87	BUC	1001	2345	1
1235	POPESCU ION	2	121B	02-04-89	TARGU-JIU	1002	1300	1
1236	AVRAM NICOLAE	1	115A	21-03-68	TARGU-JIU	1001	3000	2
1237	IONESCU MARIANA	2	116C	05-05-89	BUC	1002	1234	3
1256	POPESCU GINA	3	114A	06-09-90	TARGU-JIU	1003	3456	2

Tabela SPEC

CODS	NUME	DOMENIU
1	AUTOMATICA	CALCULATOARE
2	ENERGETICA	INGINERIE ELECTRICA
3	MECANICA	INGINERIE MECANICA

Tabela BURSA

Bursa de exceptie	2500	3999	300
Tip	Pmin	Pmax	Suma
Fara bursa	0	399	
Bursa sociala	400	899	100
Bursa de studiu	900	1799	150
Bursa de merit	1800	2499	200

**SELECT [DISTINCT] lista_de_expresii
FROM nume_tabela**

WHERE conditie_linie

-- clauza optionala **ORDER**

BY criterii_sortare_rezultat;

-- clauza optionala

EFFECT

Se parcurg rând pe rând liniile tabelii specificate pe clauza **FROM**.

Din fiecare linie conținând date pentru care condiția aflată pe clauza **WHERE** este adevărată va rezulta o linie în rezultatul cererii.

În cazul în care **WHERE** lipsește, toate liniile tabelii **FROM** vor avea o linie corespondentă în rezultatul cererii.

Linia de rezultat este compusă pe baza listei de expresii aflată pe clauza **SELECT**.

Dacă exista cuvântul cheie **DISTINCT**, din rezultat se elimină liniile duplicate. Înainte de a trimite rezultatul, serverul îl sortează în funcție de criteriile specificate de clauza **ORDER BY**.

În cazul în care **ORDER BY** lipsește, liniile din rezultat sunt într-o ordine independentă de conținutul lor sau de ordinea în care ele au fost adăugate în tabelă.

REZULTAT

Numărul coloanelor din rezultat este egal cu numărul expresiilor din lista aflată pe clauza **SELECT**.

Aceste expresii dau și numele coloanelor din rezultat.

În lipsa clauzei **DISTINCT**, numărul de linii din rezultat este egal cu numărul liniilor din tabelă care îndeplinesc condiția **WHERE** sau, când clauza respectivă lipsește, cu numărul total de linii din tabelă.

Evaluarea valorii de adevăr a condiției din **WHERE** se face doar pe baza datelor aflate pe linia respectivă.

Deoarece parcurgerea liniilor specificată de o cerere **SELECT** se face după un plan de execuție generat de server, folosirea clauzei **ORDER BY** este obligatorie în cazul în care se dorește un rezultat sortat după anumite criterii.

LISTA SELECT

a. Nume de coloane sau *

Exemplu 1:

```
SELECT NUME, DOMENIU  
FROM SPEC;
```

Exemplu 2:

```
SELECT *  
FROM STUD;  
LISTA SELECT
```

b. Constante:

Exemplu 3:

```
SELECT 'Specializarea ', NUME, ' infiintata in ', 1995  
FROM SPEC  
LISTA SELECT
```

c. Expresii aritmetice:

Exemplu 4:

```
SELECT TIP, SUMA, (SUMA+20)*1.1  
FROM BURSA;  
LISTA SELECT
```

d. Expresii concatenate:

Exemplu 5:

```
SELECT 'Specializarea ' || NUME || ' are codul ', CODS  
FROM SPEC;
```

Exemplu 6:

Cu valori nule:

```
SELECT TIP, ' are valoarea ' || SUMA || '.Lei'  
FROM BURSA;  
LISTA SELECT
```

e. Alias de coloana:

Nu poate fi mai lung de 30 de caractere.

Începe cu o litera, contine decât litere, cifre, `_`, `#` si `$` sau e pus între ghilimele (tot max. 30 caractere între ghilimele).

Între ghilimele literele mici sunt considerate diferite de literele mari.

Nu poate fi folosit decât în cererea curentă.

Sistemul nu stochează în baza de date sau altundeva aceste nume alternative.

Nu poate fi folosit în alte clauze ale cererii (doar în **SELECT** și **ORDER BY**).

f. Alias de coloana:

Exemplu 7:

```
SELECT TIP AS "Tip bursa", ' are valoarea ' || SUMA || '.Lei' AS Descriere  
FROM BURSA;
```

Rezultat:

Tip bursa	DESCRIERE
-----	-----
FARA BURSA are	valoarea .Lei
BURSA SOCIALA are	valoarea 100.Lei

g. DISTINCT: Elimina liniile duplicat din rezultat:

Exemplu 8:

```
SELECT CODS  
FROM STUD;
```

Exemplu 9:

```
SELECT DISTINCT CODS  
FROM STUD;
```

Exemplu 10:

```
SELECT DISTINCT CODS, AN  
FROM STUD;
```

h. CLAUZA WHERE

Sintaxa:

WHERE expresie_logica

Exemplu 11:

```
SELECT NUME, GRUPA, CODS  
FROM STUD  
WHERE AN = 4;
```

f1. CLAUZA WHERE - Operatori de comparare

Conditii compuse (**AND, OR, NOT**) și paranteze

AN=2 AND PUNCTAJ>500 OR CODS=11

AN=2 AND (PUNCTAJ>500 OR CODS=11)

f2. CLAUZA WHERE - Operatorul BETWEEN

Sintaxa:

expresie **BETWEEN** valoare_minima **AND** valoare_maxima

Exemplu 12:

```
SELECT NUME, AN, PUNCTAJ  
  
FROM STUD  
WHERE PUNCTAJ BETWEEN 2000 AND 4000;  
CLAUZA WHERE
```

Alte exemple

Exemplu 13:

```
SELECT NUME, AN, PUNCTAJ  
FROM STUD  
WHERE PUNCTAJ + 100 BETWEEN TUTOR - 2000 AND TUTOR + 1000;
```

Exemplu 14:

```
SELECT NUME, LOC, DATAN  
FROM STUD  
WHERE LOC BETWEEN 'A' AND 'L' AND DATAN  
      BETWEEN '1-JAN-82' AND '31-DEC-82';
```

f3. CLAUZA WHERE - Operatorul IN:

Sintaxa:

expresie **IN** (val_1, val_2, ..., val_n)

Exemplu 15:

```
SELECT NUME, AN, DATAN  
FROM STUD  
WHERE TUTOR IN (1456, 2146);
```

1. IN ignora valorile nule din listă:

Exemplu 16:

```
SELECT NUME, AN, GRUPA, TUTOR  
FROM STUD
```

WHERE TUTOR IN (NULL, 1456, 2146);

2. NOT IN intoarce fals dacă lista conține valori nule:

Exemplu 17:

SELECT NUME, AN, GRUPA, TUTOR
FROM STUD
WHERE TUTOR **NOT IN** (NULL, 1456, 2146);

3. IN este operator derivat:

Exemplu 18:

SELECT NUME, AN, DATAN
FROM STUD
WHERE TUTOR=1456 **OR** TUTOR=2146;
CLAUZA WHERE

4. Operatorul IN.

Exemplu 19:

SELECT NUME, PUNCTAJ, CODS
FROM STUD
WHERE PUNCTAJ + 10 **IN** (CODS*30+70, CODS*200+700);

Exemplu 20:

SELECT NUME, LOC, DATAN
FROM STUD
WHERE LOC **IN** ('BUCURESTI', 'PLOIESTI')
OR DATAN **IN** ('02-SEP-85', '19-APR-84', '29-AUG-84');

